Run length encoding

What is Run Length Encoding (RLE)?

- **RLE** is a **lossless data compression technique**.
- It works by **replacing consecutive repeated characters (or symbols) with a single symbol and a count**.
- Goal: **reduce storage size** by eliminating redundancy.

Principle of RLE

Example word:

```nginx
aaaaabbccccccffwxyyy
```

- Normal length: **19 letters**
- After RLE:

```
5a2b5c2f1w1x3y
```

- Compressed length: **14 characters**

Here,

- "aaaaa" → `5a`
- "bb" → `2b`
- "ccccc" → `5c`

  and so on.

So, RLE is effective when **long runs of the same symbol** appear.

Limitation

Not always efficient.

Example: word **Cat**

- Original: `Cat` → 3 characters
- Encoded: `1c1a1t` → 6 characters

RLE can actually **increase data size** if data has **low repetition**.

Applicability (Images)

- Works best on **binary images** (black & white) or images with large areas of same color.
- Example: **35 × 35 image** (binary, i.e., only black/white).

Instead of storing each pixel separately (1225 bits),
we store runs:

```nginx
n1w n2b n3w ...
```

Where:

- `w` = white
- `b` = black
- `n1`, `n2`, … = number of consecutive pixels of that color.

Example row:

```
5w3b15w3b9w
```

Meaning: 5 white pixels, 3 black, 15 white, 3 black, 9 white.

This drastically reduces storage.

One Implementation Technique

- Encode each **row of image pixels** separately.
- Store alternating runs of **white (w)** and **black (b)** pixels.
- This creates compact representation.

Usefulness

- Very efficient when data has **lots of repetition** (e.g., fax documents, simple graphics, icons, scanned text images).
- Not effective for complex/colorful images with **less redundancy**.

Summary Table

Feature	Explanation
Type	Lossless compression
Method	Replace runs of repeated symbols with count + symbol
Best For	Data/images with long sequences of identical values
Limitation	Can increase size when repetition is low
Use Cases	Fax, icons, simple bitmaps, text with repeating characters

Data compression

- Source coding $\equiv$ Data compression $\equiv$ Bit-rate reduction

Principle of RLE

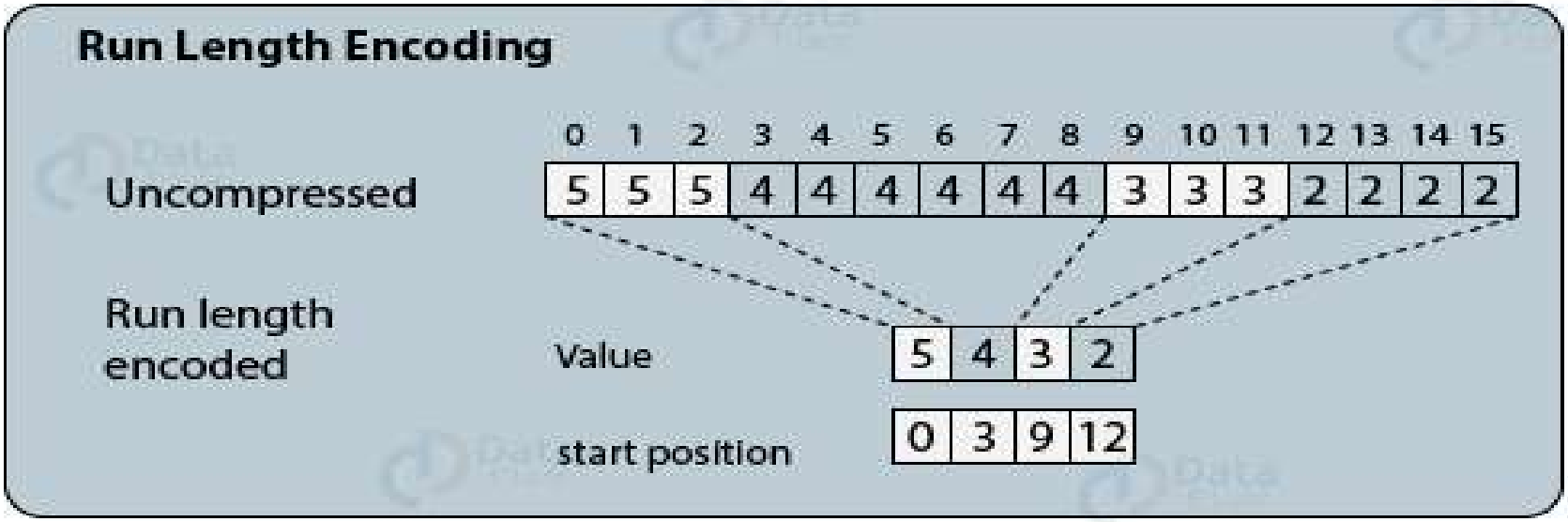
- Assume a 19 letter word aaaaabbccccffwxyyy
- 5a2b5c2f1w1x3y
  - Becomes 14 letter word

Limitation

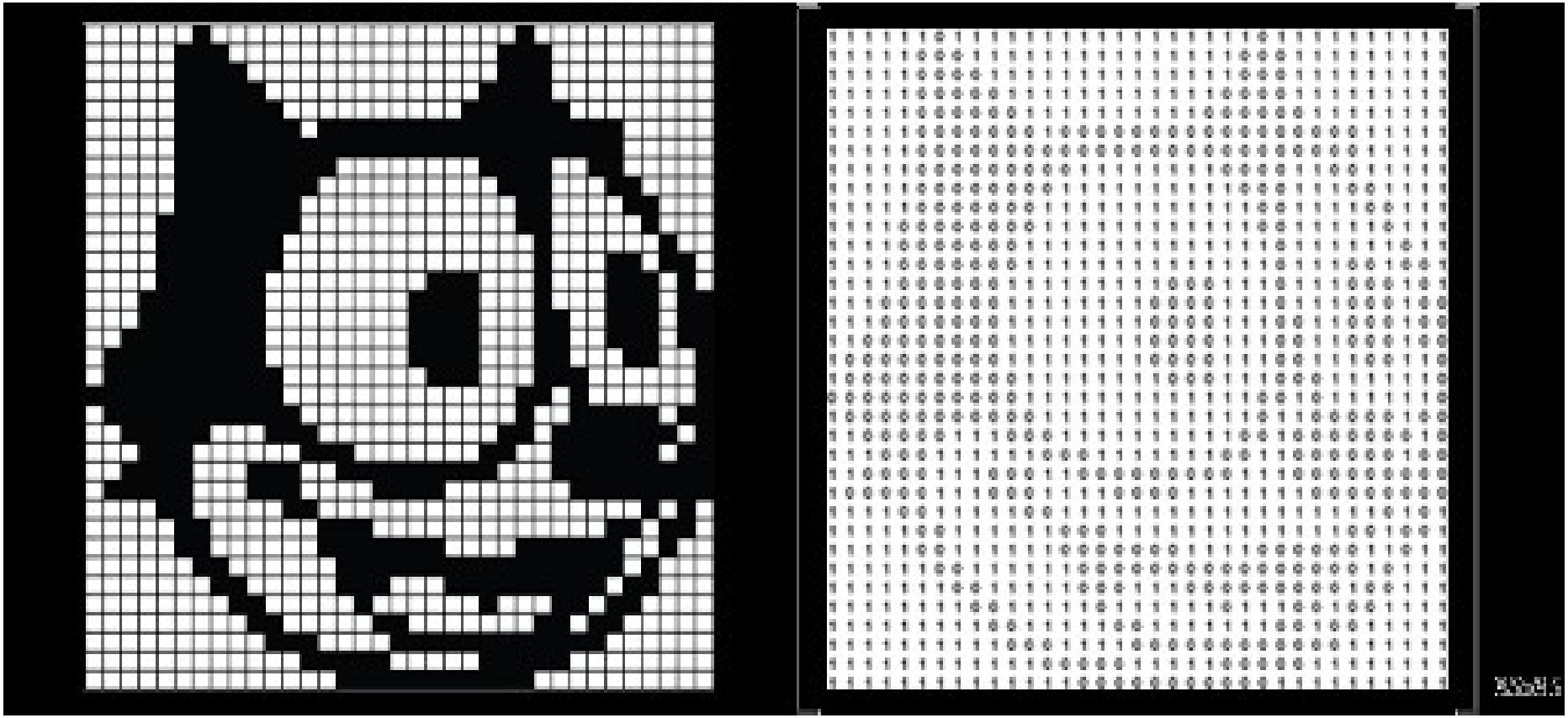
- Cat – 3 letter word
- 1c1a1t – 6 letter word

Applicability

- Images

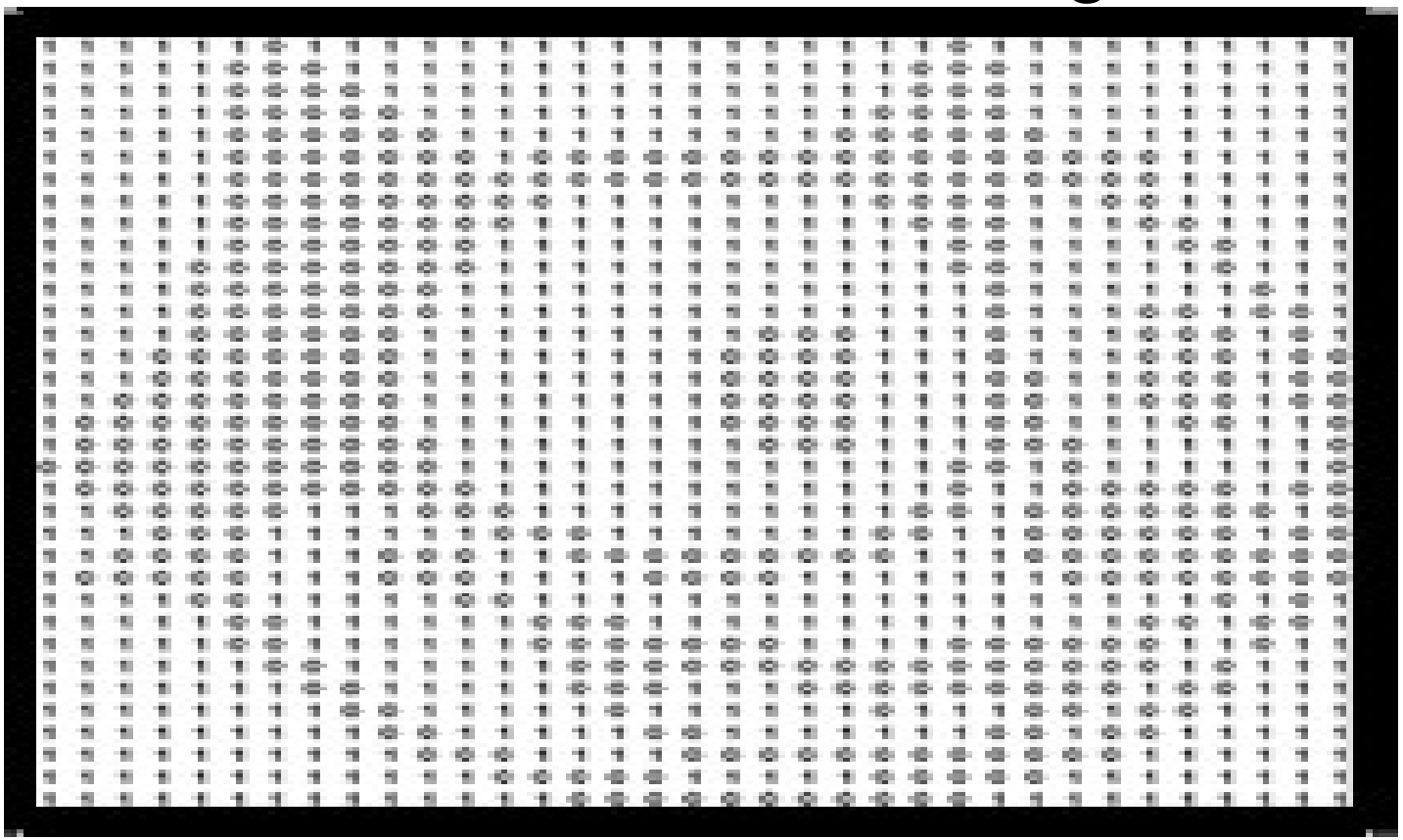


35 x 35 image



- 2 symbols
- 35 X 35 = 1225 bits needed to store

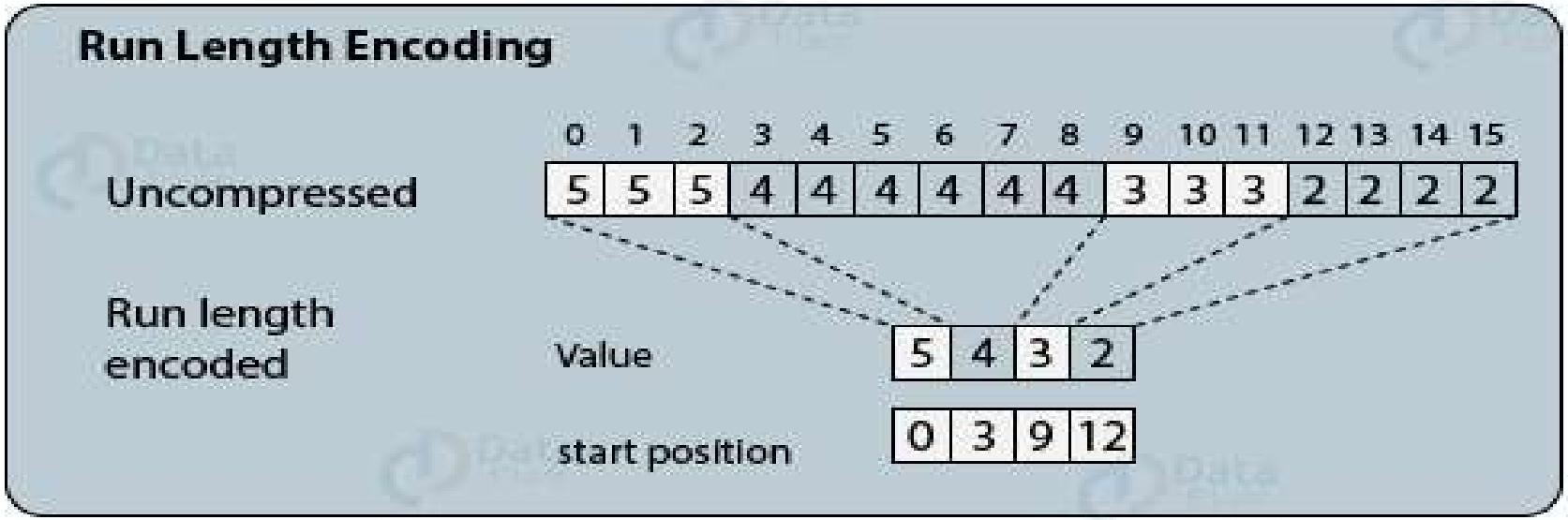
35 x 35 image



- $n_1wn_2bn_3wn_4bn_5w$ – 1st row over
- 5w3b15w3b9w – 2nd row over
- ….

1	1	1	1	1	1	1	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	0	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	0	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	0	1	1	0	1	1	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	0	1	1	1	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	0	1	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	1	1
1	0	1	1	0	1	1	1	1	0	0	0	1	1	0	0	0	0	1	1	1	1	1	1	1
0	0	0	0	1	1	1	1	0	0	0	1	1	0	1	1	0	0	1	1	1	1	1	1	1
0	1	1	0	1	1	1	0	0	0	0	1	0	1	1	0	1	0	1	1	1	1	1	1	1
0	1	1	0	1	1	0	0	0	1	0	0	1	1	0	1	1	0	1	1	1	1	1	1	1
0	1	1	0	1	0	0	0	1	1	0	1	1	0	1	1	1	0	1	1	1	1	1	1	1
0	1	1	0	0	0	0	1	1	0	0	1	0	1	1	1	1	0	1	1	1	1	1	1	1
0	1	1	0	1	1	0	1	0	1	0	0	1	1	1	1	0	1	1	1	1	1	1	1	1
0	1	1	0	1	1	0	0	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1
0	1	1	0	0	0	0	0	0	0	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

One implementation technique



Usefulness

- Efficient with files that contain lots of repetitive data